Backend Server -> FoodOrderingServer
Frontend Client -> FoodOrderingClient app1, app2

- npm create vite app1

JSX -> Javascript and XML
TSX -> Typescript and XML

Components in JSX
- These are reusable entities
- Components are created using Functions which can be called multiple times
- These functional components can be also accessed using the way similar like html elements
- Rules to design functional component
1. The name of the functionla component must be in pascal case
2. It must return an jsx

https://marketplace.visualstudio.com/items?itemName=NuclleaR.vscode-extension-auto-import
https://marketplace.visualstudio.com/items?itemName=rodrigovallades.es7-react-js-snippets

Creating React application
Components
types
pages
event handling
props -> {}()
state

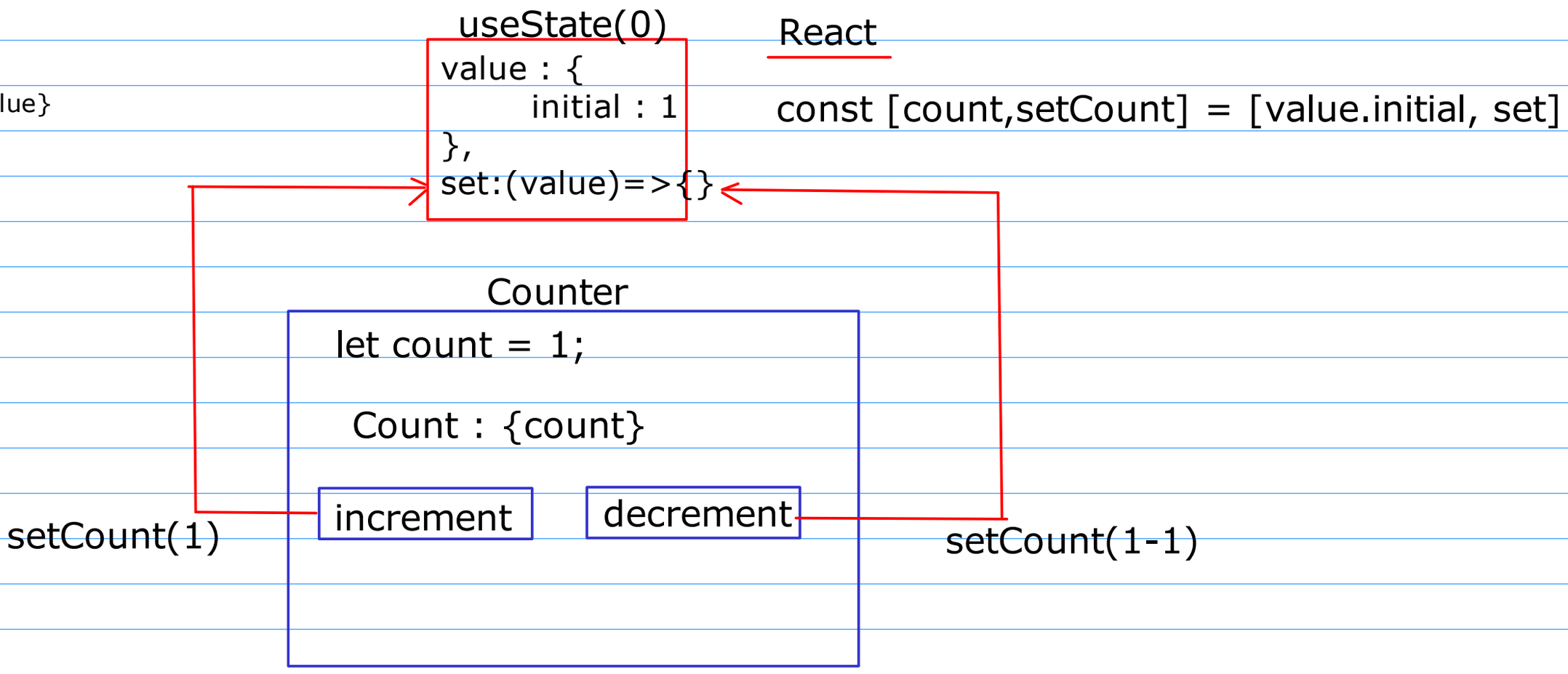
Steps to work with project

- create a new app4 project
    npm create vite app4
- install the node modules
    cd app4
    npm install
- run the application
    npm run dev
- clear the App.css and index.css files
- from main.tsx file remove the StrictMode and clear the unwanted imports
- in App.tsx delete the entire code and create the blank Functional Component(App) once again

#hooks
- functions in react that starts with the prefix use
- useState()

state
- It is a way of creating the local variables for component by the react
- It is entirely managed by the react which helps to perform rerendeing of the UI when changed

```
set : (value)=>{
return {initial:value}
}
```



Steps to add tailwind in your project
- complete the steps for your new Project
- install the tailwindcss using below command
    npm install tailwindcss @tailwindcss/vite
- edit the vite.config.ts file and add the tailwindcss() plugin in it
- inside the index.css file import the tailwindcss
    @import "tailwindcss";
- to add daisyUI(component library) use below command
    npm install daisyui
- add the daisyui plugin in the index.css file below tailwind
    @plugin "daisyui";

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react(), tailwindcss()],
})
```

main-> Program

```
UserService us = new UserService();
us.addUser();
```

service -> UserService.java
(Business Logic)

```
public void addUser(){
        Scanner sc = new Scanner(System.in);
        User user = new User();
        sysout("Enter name");
        user.setname(sc.next());
        UserDao userDao = new userDao();
        userDao.insertUser(user);
}
```

dao -> UserDao.java

Database Operations

```
public void insertuser(User user){
sql = "Insert into....."
insert statemnt= connection.prepareStatement()
insertStatement.executeUpdate();
}
```

An async function in JS always return the result wrapped in promises